

Teste de Software – Test Pattern

Padrões de Criação de Dados (Builders)

Porque usar CarrinhoBuilder em vez de CarrinhoMother

O objeto Carrinho é mais complexo do que um simples User, pois ele pode variar em:

- Tipo de usuário (padrao ou premium)
- Quantidade e valores dos itens
- Estado (vazio, com um item, com vários itens)

Um CarrinhoMother exigiria muitos métodos estáticos para cobrir todas as combinações possíveis, por exemplo:

- CarrinhoVazio()
- CarrinhoComUmItemPadrao()
- CarrinhoPremiumComItensCaros()
- CarrinhoPadraoComItensBaratos()

Essa quantidade de métodos aumenta o acoplamento entre testes e fábrica de dados. Com o tempo, a manutenção fica cara e o código de teste perde clareza.

O CarrinhoBuilder resolve isso ao fornecer:

- Valores padrões simples
- Métodos fluentes para alterar apenas o que importa
- Controle explícito do estado que o teste precisa

Exemplo Antes (setup manual complexo)

No setup manual, o teste precisa conhecer detalhes de construção e repetir código. Isso aumenta o ruído e o risco de erros.

```
const user = new User(2, 'Usuario Premium', 'premium@email.com', 'PREMIUM');
const itens = [
  new Item('Item A', 120),
  new Item('Item B', 80),
];
const carrinho = new Carrinho(user, itens);
```

Se outro teste precisar de um carrinho vazio ou com outro usuário, o trecho acima precisa ser reescrito e ajustado.

Exemplo Depois (setup com Data Builder)

Com o CarrinhoBuilder, o teste fica mais legível e explícito sobre o que está sendo modificado.

```
const carrinho = new CarrinhoBuilder()
  .comUser(UserMother.umUsuarioPremium())
  .comItens([
    new Item('Item A', 120),
    new Item('Item B', 80),
  ])
  .build();
```

Se for necessário um carrinho vazio, a intenção fica evidente:

```
const carrinhoVazio = new CarrinhoBuilder().vazio().build();
```

Como o Builder melhora legibilidade e manutenção

- **Legibilidade:** o teste mostra apenas as variações relevantes do cenário. O leitor entende rapidamente o que é diferente naquele caso.
- **Manutenção:** mudanças na estrutura do Carrinho ficam concentradas no builder, reduzindo ajustes repetidos em vários testes.
- **Evolução segura:** é fácil adicionar novos métodos fluentes sem quebrar testes existentes.

Esse uso previne o Test Smell de Setup Obscuro, porque o setup fica enxuto e claro.

Padrões de Test Doubles (Mocks vs Stubs)

Teste escolhido: sucesso Premium

O teste de sucesso para usuário premium está em `tests/CheckoutService.test.js`. Nele, as dependências são tratadas assim:

- GatewayPagamento -> Stub
- PedidoRepository -> Stub
- EmailService -> Mock

Porque GatewayPagamento foi um Stub

O objetivo principal do GatewayPagamento no teste é controlar o fluxo do CheckoutService. Precisamos simular o retorno `{ success: true }` para permitir que o processo continue e gere o pedido.

Ou seja, o foco está no estado resultante do método `processarPedido` (pedido salvo e email enviado). O stub serve para dirigir o caminho do código sem depender do gateway real.

Apesar de o teste verificar se cobrar recebeu o valor correto, a dependência ainda é usada principalmente como stub porque o retorno dela é o que determina o fluxo principal. A verificação do valor é um reforço da regra de negócio (desconto premium), não o objetivo central da dependência.

Porque EmailService foi um Mock

O EmailService representa um efeito colateral externo (envio de email). Nesse caso, o teste precisa verificar o comportamento, isto é, se a chamada aconteceu e com quais argumentos.

Assim, o teste valida:

- Se enviarEmail foi chamado
- Quantas vezes foi chamado
- Quais parâmetros foram usados

Esse é o papel clássico de um mock: verificação de comportamento. O retorno do EmailService não muda o caminho do código, mas a chamada é importante para confirmar o efeito colateral esperado.

Verificação de Estado vs. Comportamento

- **Stubs**: usados para controlar o estado e o fluxo do teste (retornos pré-definidos).
- **Mocks**: usados para verificar interações (métodos chamados com parâmetros corretos).

No teste premium, o GatewayPagamento e o PedidoRepository são stubs porque fornecem dados que permitem a evolução do fluxo. O EmailService é mock porque o foco está em verificar a chamada.

Conclusão

O uso deliberado de Builders e Test Doubles torna a suíte de testes mais clara e sustentável. Builders reduzem duplicação e deixam explícito o que varia em cada cenário, evitando o Test Smell de Setup Obscuro. Já a separação entre stubs e mocks ajuda o teste a focar no que importa: estado final quando o objetivo é resultado, e comportamento quando o objetivo é efeito colateral.

Esses padrões reduzem acoplamento, facilitam manutenção e deixam os testes mais confiáveis ao longo do tempo. Isso contribui diretamente para uma base de testes que cresce de forma segura e compreensível.